



National Institute of Applied
Science and Technology

Carthage University

Academic Project
Department of Chemical Engineering
Winter 2025

Technical Report

Savitsky-Golay Filtering Method

Student:

Sofien BOUTALEB

Fourth-year Engineering Student

Material and Process Simulation Enthusiast

2025-02-18

Contents

Contents	1
Table of Figures	2
1. Introduction	3
2. Theoretical Background	4
2.1. Polynomial Fitting	4
2.2. A Practical Example with High Noise	7
2.3. Convolution Coefficients	10
3. Applications.....	10
4. Choosing Parameters	11
5. Batch Process Optimization Using Savitzky-Golay Filter in Chemical Engineering.....	12
5.1 Problem Overview	12
5.2 Assumed Data.....	12
5.3. Python Code Implementation with Experimental Data.....	13
5.4. Interpretation of Results:.....	14
6. Conclusion	18
7. Reference	19

Table of Figures

Figure 1 Example on filtering effects.....	3
Figure 2 Polynomial Smoothing with a Sliding Window	6
Figure 3 Polynomial Fitting: Underfitting vs. Overfitting with High Noise Levels	9
Figure 4 Original vs. Smoothed Concentration of A Over Time	14
Figure 5 Rate of Change of Concentration (Before and After Filtering)	15

1. Introduction

The Savitsky-Golay filter is a method used for smoothing noisy data while preserving key features such as peaks and derivatives. It is based on fitting a polynomial to a local subset of data points using the least squares method. This technique is particularly useful in spectroscopy, time-series analysis, and signal processing Savitsky smoothing 1964.

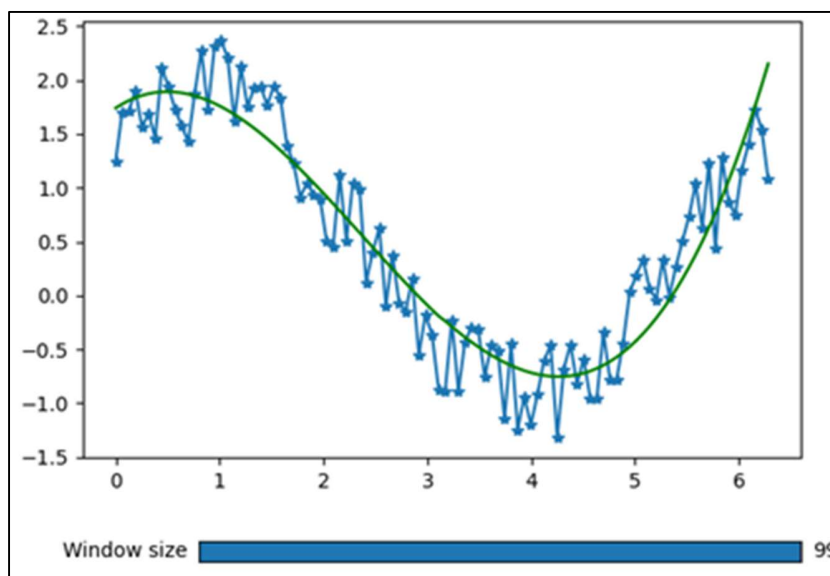


Figure 1 Example on filtering effects.

The Savitsky-Golay filter is crucial in chemical engineering for smoothing noisy experimental and process data while preserving key features like peak heights and widths. It is widely used in **spectroscopy (UV-Vis, IR, Raman, NMR)** to enhance signal clarity, in **process monitoring** to filter real-time sensor data, and in **reaction kinetics** for accurate derivative calculations. Additionally, it aids in **rheology and material characterization** by refining viscosity and stress-strain measurements, improves environmental monitoring and quality control by reducing noise in analytical data, and enhances computational simulations by clarifying numerical trends. This makes it an essential tool for precise data interpretation in research and industry.

That's why we aim to undertake a brief project on the Savitzky-Golay method: to explore its capabilities in data smoothing and differentiation, thereby enhancing the precision and reliability of data analysis in complex scientific and engineering applications.

2. Theoretical Background

Savitsky-Golay filtering is based on the least-squares polynomial approximation within a moving window. Given a set of data points, the method fits a polynomial of degree d within a sliding window of size w and uses its central point as the smoothed value.

2. 1. Polynomial Fitting

For a given window size w and polynomial order d , the filter minimizes the squared difference between the original data and the fitted polynomial.

In this essay, we will explore the core principles behind polynomial fitting, its implementation, and one of its most powerful applications: the Savitzky-Golay method for smoothing and differentiation.

Understanding Polynomial Fitting

At its core, polynomial fitting involves finding the polynomial function that best approximates a set of data points. A polynomial is an expression consisting of variables raised to integer powers and multiplied by coefficients. In the case of fitting, the polynomial is typically represented as:

$$P(x) = a_0 \cdot x_0 + a_1 \cdot x_1 + \dots + a_n \cdot x_n$$

Where $P(x)$ is the polynomial function, $a_0, a_1, \dots, a_n$ are the coefficients to be determined, and x is the independent variable. The goal of polynomial fitting is to find the values of the coefficients $a_0, a_1, \dots, a_n$ that minimize the difference between **the polynomial curve** and **the data points**.

The Least Squares Method

One of the most common methods for fitting a polynomial to data is the **least squares method**. This method minimizes the sum of the squared differences between the observed values (data points) and the values predicted by the polynomial. The least squares method provides an optimal solution for fitting a polynomial, and it is widely used due to its simplicity and efficiency.

The error between the observed values y_i and the predicted values $P(x_i)$ at each point x_i is given by:

$$E = \sum_{i=1}^N (y_i - P(x_i))^2$$

The objective is to find the polynomial coefficients that minimize this error function. Once the coefficients are found, the resulting polynomial represents the best fit to the data, according to the least squares criterion.

Process:

1. **Moving Window:** A fixed-size window (e.g., 5 or 7 data points) moves sequentially across the dataset, selecting a subset of points at each step.
2. **Polynomial Fitting:** A low-degree polynomial (often quadratic or cubic) is fitted to the selected data points, ensuring the best approximation using the least-squares criterion.
3. **Least-Squares Criterion:** The polynomial coefficients are determined by minimizing the sum of squared differences between the fitted polynomial values and actual data points.
4. **Sliding Window:** Once a polynomial is fitted, the window moves forward by one data point, and the process repeats. The polynomial values at each step form a smoothed representation of the data.

Example:

Given the following data points:

x	1	2	3	4	5
Y	2	3	5	6	5

1. **Window Selection:** Using a window of 3 points, we fit a quadratic polynomial

$$y = a.x^2 + b.x + c.y$$

2. **First Fit (Points 1,2,3):** Solve the system:

$$\begin{cases} a + b + c = 2 \\ 4.a + 2.b + c = 3 \\ 27.a + 3.b + c = 5 \end{cases} \quad \text{Solving for a,b,c we get:} \quad \begin{cases} a = \frac{1}{2} \\ b = \frac{-1}{2} \\ c = 2 \end{cases}$$

So, we get:

$$y = \frac{1}{2}x^2 + \frac{1}{2}x + 2$$

3. **Second Fit (Points 2,3,4):** Repeat the process for (2,3), (3,5), and (4,6).
4. **Third Fit (Points 3,4,5):** Repeat for (3,5), (4,6), and (5,5).

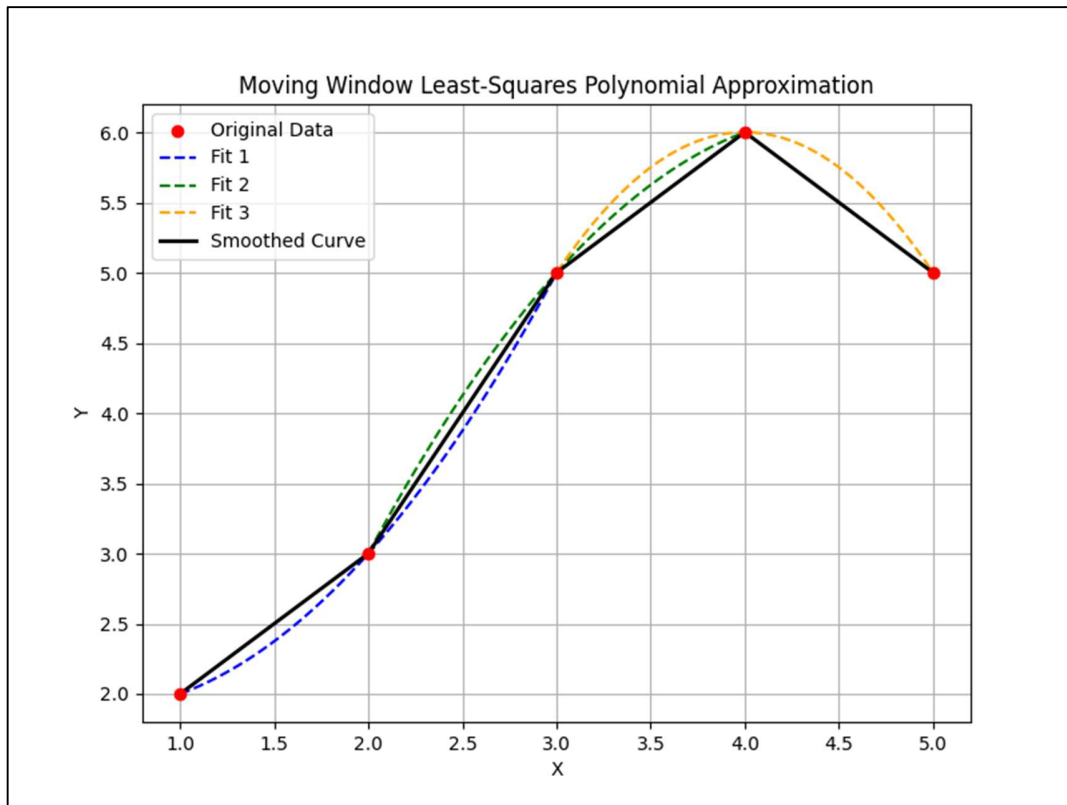


Figure 2 Polynomial Smoothing with a Sliding Window

Conclusion:

This iterative approach provides a smooth approximation while preserving the underlying trend of the data. The resulting curve reduces noise and enhances trend visualization, making it a useful tool for signal processing, time series analysis, and data smoothing.

Polynomial Smoothing with a Sliding Window.

Polynomial Degree and Overfitting

The degree of the polynomial (n) plays a crucial role in polynomial fitting. A low-degree polynomial might **fail** to capture the underlying trend of the data, while a high-degree polynomial may lead to **overfitting**.

Overfitting occurs when the polynomial curve fits the noise in the data rather than the actual trend, making the model less generalizable to new data. To avoid overfitting, it is essential to choose an appropriate polynomial degree that balances complexity and accuracy.

2. 2. A Practical Example with High Noise

In this example, we will explore the impact of polynomial fitting with noisy data. The key concept is that the degree of the polynomial we choose has a profound effect on how well the model captures the data. A low-degree polynomial might fail to capture the trend, while a high-degree polynomial might start fitting the noise itself, a phenomenon called **overfitting**.

We'll demonstrate this using a **highly noisy dataset**. To really see the difference between underfitting and overfitting, I added a lot of random noise to our data points. This will show how a polynomial of a low degree struggles to capture the true underlying pattern, while a polynomial of a higher degree follows even the random fluctuations in the noise itself.

The Code Breakdown

1. **True Function:** First, we generate a simple quadratic function (exp: $3x^2 + 2x + 1$) that represents the true underlying trend of the data.
2. **Noisy Data:** Then, we add significant noise to this function by randomly perturbing the values. This noise will make the trend harder to discern.
3. **Polynomial Fits:** We fit two polynomials to the noisy data:
 - One with a **low degree (2)**, which may not capture the data trend accurately.
 - One with a **high degree (10)**, which might overfit the noise and create a very wiggly curve.

The Code using python:

```
import numpy as np
import matplotlib.pyplot as plt

# Generate some data: True quadratic function without noise
def true_function(x):
    return 3 * x**2 + 2 * x + 1

# Generate highly noisy data with fewer points
np.random.seed(0) # For reproducibility
x = np.linspace(0, 10, 10) # Reduced number of points
y_true = true_function(x)

# Add much higher noise to the data to make the difference clear
```



```
y_noisy = y_true + np.random.normal(0, 50, size=x.shape) # High
noise level

# Polynomial degrees to demonstrate the effect
degrees = [2, 10] # Low degree (underfitting) and high degree
(overfitting)

# Create a plot
plt.figure(figsize=(12, 6))

# Plot original true function
plt.plot(x, y_true, label='True Function:  $3x^2 + 2x + 1$ ',
color='blue', linewidth=2)

# Plot noisy data
plt.scatter(x, y_noisy, color='black', label='Highly Noisy Data',
zorder=5)

# Loop through different degrees
for degree in degrees:
    # Fit polynomial to the noisy data
    coeffs = np.polyfit(x, y_noisy, degree)
    poly = np.poly1d(coeffs)

    # Generate y values for the fitted polynomial
    y_fit = poly(x)

    # Plot the fitted polynomial
    plt.plot(x, y_fit, label=f'Polynomial Degree {degree}')

# Add labels and title
plt.xlabel('X')
plt.ylabel('Y')
plt.title('Polynomial Fitting: Underfitting vs Overfitting (High
Noise)')
plt.legend()
plt.grid(True)

# Show the plot
plt.show()
```

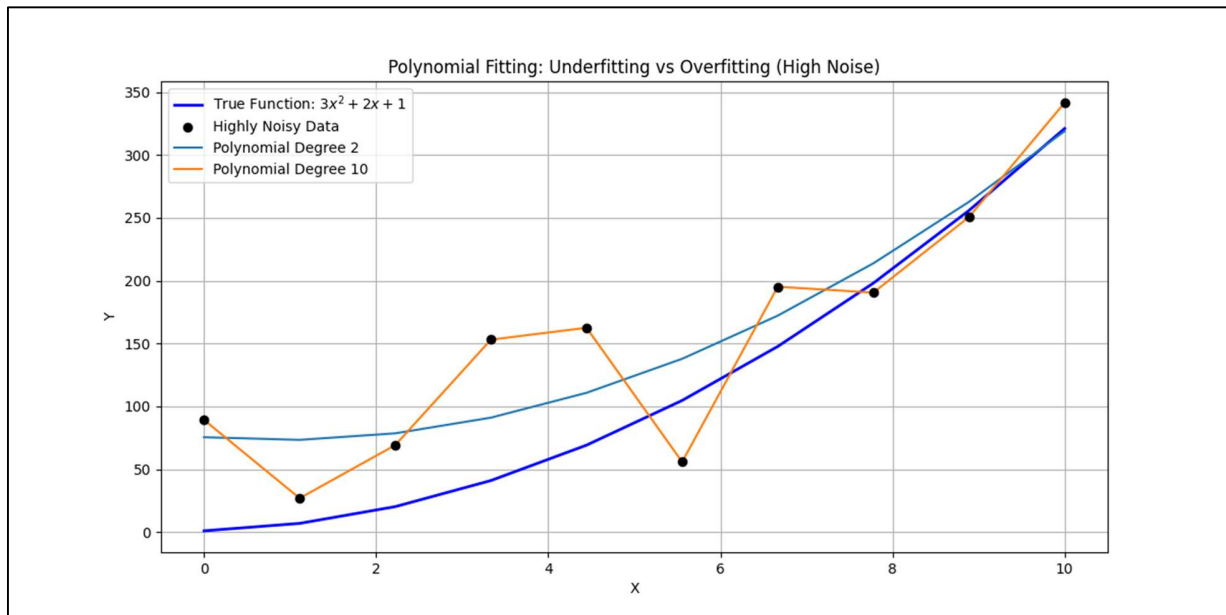


Figure 3 Polynomial Fitting: Underfitting vs. Overfitting with High Noise Levels

Explanation of the Results

1. **True Function (Blue Line):** The blue line represents the true relationship in the data, which is a quadratic function. It's smooth and represents the actual pattern we are trying to capture.
2. **Noisy Data (Black Dots):** The black dots represent the noisy data. As you can see, it doesn't follow the smooth curve of the true function due to random fluctuations added to the data.
3. **Degree 2 Polynomial (Red Line):** The polynomial of degree 2 fits the noisy data but doesn't follow the fluctuations. It captures the general trend but fails to fit the noise exactly. This is a classic example of **underfitting**, where the model is too simple to capture the complexity of the data.
4. **Degree 10 Polynomial (Green Line):** The polynomial of degree 10 fits the data almost perfectly but notice how it starts to follow the noise as well, creating a very jagged curve. This is an example of **overfitting**—the model fits the noise in the data, making it less generalizable and potentially less accurate for new data points.

Conclusion

By increasing the noise level in the data, the differences between the polynomial fits become much more noticeable. With fewer data points, the high-degree polynomial tries to fit every

single fluctuation, creating a curve that is highly sensitive to the noise. On the other hand, the lower-degree polynomial (degree 2) provides a smoother fit that captures the main trend but ignores the noise.

This example illustrates why it is crucial to choose an appropriate polynomial degree. Too low a degree might miss important trends (underfitting), and too high a degree can lead to overfitting, where the model is too complex and doesn't generalize well. Balancing these two aspects is key to making accurate predictions with polynomial fitting.

2.3. Convolution Coefficients

Convolution Coefficients are the values that define how a filter modifies the data during convolution. In methods like the **Savitzky-Golay filter**, these coefficients are precomputed using polynomial fitting techniques to ensure the filter optimally fits the data. Once computed, the coefficients are used to efficiently smooth or process the data by applying a convolution operation. This precomputation leads to significant computational savings, especially in real-time signal processing or when working with large datasets.

By precomputing these coefficients, we can focus on applying the filter to various data without repeatedly recalculating the filter parameters, making the overall process much faster and more efficient.

3. Applications

Savitzky-Golay filters are used in various fields, including:

The **Savitzky-Golay filter** is a widely used technique for smoothing noisy data and preserving important features. Here are some key applications of this method:

- **Signal Processing**

Used for reducing noise in signals while retaining important features such as peaks and valleys. It's applied in audio signal processing, for example, to remove noise from recordings.

- **Time Series Analysis**

Helps smooth time series data and identify trends in volatile data, such as stock prices or currency fluctuations. It is also used for computing derivatives of time-series data, like velocity from position.

- **Spectral Analysis**

Commonly used in spectroscopy and chromatography to clean noisy spectra, enabling clearer identification of absorption peaks and improving material analysis.

- **Biomedical Engineering**

Used in processing ECG or EEG signals to remove noise, making it easier to analyze heart rate or brainwave patterns. It is also applied in medical imaging to reduce noise in MRI and CT scans.

- **Environmental Sciences**

Helps smooth meteorological and geophysical data, such as temperature readings or seismic signals, to reveal long-term trends and detect anomalies.

- **Manufacturing and Quality Control**

In manufacturing, it is used for analyzing sensor data to detect defects or inconsistencies in machines. It also helps smooth vibration data to monitor the health of machinery.

- **Robotics and Control Systems**

Applied in navigation systems to smooth sensor data, such as accelerometer or gyroscope readings, for more accurate motion predictions and trajectory estimation.

- **Astronomy**

Used in astronomical signal processing to smooth data from telescopes, helping astronomers better detect subtle changes in light curves or other signals from celestial objects.

4. Choosing Parameters

- The effectiveness of the filter depends on:

1. Window size (w): Larger windows provide smoother results but may distort sharp features.
2. Polynomial order (d): Higher orders fit more complex features but may introduce artifacts.
3. Boundary conditions: Handling edges using padding techniques or truncated windows.

5. Batch Process Optimization Using Savitzky-Golay Filter in Chemical Engineering

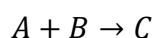
In **batch processing** in the chemical industry, several stages are involved where raw materials are mixed, reacted, and then separated to obtain a final product.

During this process, multiple variables like temperature, pressure, concentration, and flow rates need to be monitored carefully. These measurements are often noisy due to sensor imperfections, fluctuations, and environmental disturbances. To ensure that the system operates efficiently and consistently, it is essential to filter out the noise and obtain a clear understanding of the process's dynamics.

This project demonstrates how to apply the **Savitzky-Golay filter** in a batch process scenario, where we aim to optimize the reaction process by smoothing noisy data, determining concentration changes, and visualizing the results for better decision-making.

5.1 Problem Overview

Let's assume we are working with a batch reactor where a chemical reaction occurs over time:



We will measure the **concentration of reactant A** at different time intervals. However, due to noise in the sensor data, the readings fluctuate around the actual concentration.

Our goal is to:

1. Smooth the noisy concentration data.
2. Compute the rate of change (derivative) of concentration.
3. Visualize both the raw and smoothed concentration data, as well as the reaction rate.

We will use Python to demonstrate how the Savitzky-Golay filter can be applied to smooth this noisy data and analyze the batch process more effectively.

5.2 Assumed Data

We assume that the concentration of reactant A follows a smooth trend over time, but due to noise in the sensors, the data is disturbed.

We assume that we have experimental data for the concentration of reactant A at different time intervals, but due to sensor noise or other sources of error, the measurements exhibit random fluctuations. Here is an example of how the data might look after being collected from an industrial process:

Time (hours)	0	1	2	3	4	5	6	7	8	9	10
Concentration of A	1.0	0.85	0.75	0.65	0.60	0.55	0.50	0.45	0.42	0.38	0.35

5.3. Python Code Implementation with Experimental Data

The following Python script allows us to input our experimental data and apply Savitzky-Golay filtering:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.signal import savgol_filter
# Given data
time = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10])
concentration = np.array([1.0, 0.85, 0.75, 0.65, 0.60, 0.55, 0.50, 0.45, 0.42, 0.38, 0.35])
# Apply Savitzky-Golay filter
smoothed_concentration = savgol_filter(concentration, window_length=5, polyorder=2)
# Compute the numerical derivative before and after filtering
original_derivative = np.gradient(concentration, time)
smoothed_derivative = np.gradient(smoothed_concentration, time)
# Plot original and smoothed data
plt.figure(figsize=(10, 5))
# Plot concentration curves
plt.subplot(1, 2, 1)
plt.plot(time, concentration, 'o-', label='Original Data', markersize=5)
plt.plot(time, smoothed_concentration, 's-', label='Smoothed Data (Savitzky-Golay)', markersize=5)
plt.xlabel('Time (hours)')
plt.ylabel('Concentration of A')
plt.title('Original vs. Smoothed Concentration')
plt.legend()
plt.grid(True)
# Plot derivative curves
plt.subplot(1, 2, 2)
```

```
plt.plot(time, original_derivative, 'o-', label='Original  
Derivative', markersize=5)  
plt.plot(time, smoothed_derivative, 's-', label='Smoothed  
Derivative (Savitzky-Golay)', markersize=5)  
plt.xlabel('Time (hours)')  
plt.ylabel('d(Concentration)/dt')  
plt.title('Derivative Before and After Filtering')  
plt.legend()  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```

5.4. Interpretation of Results:

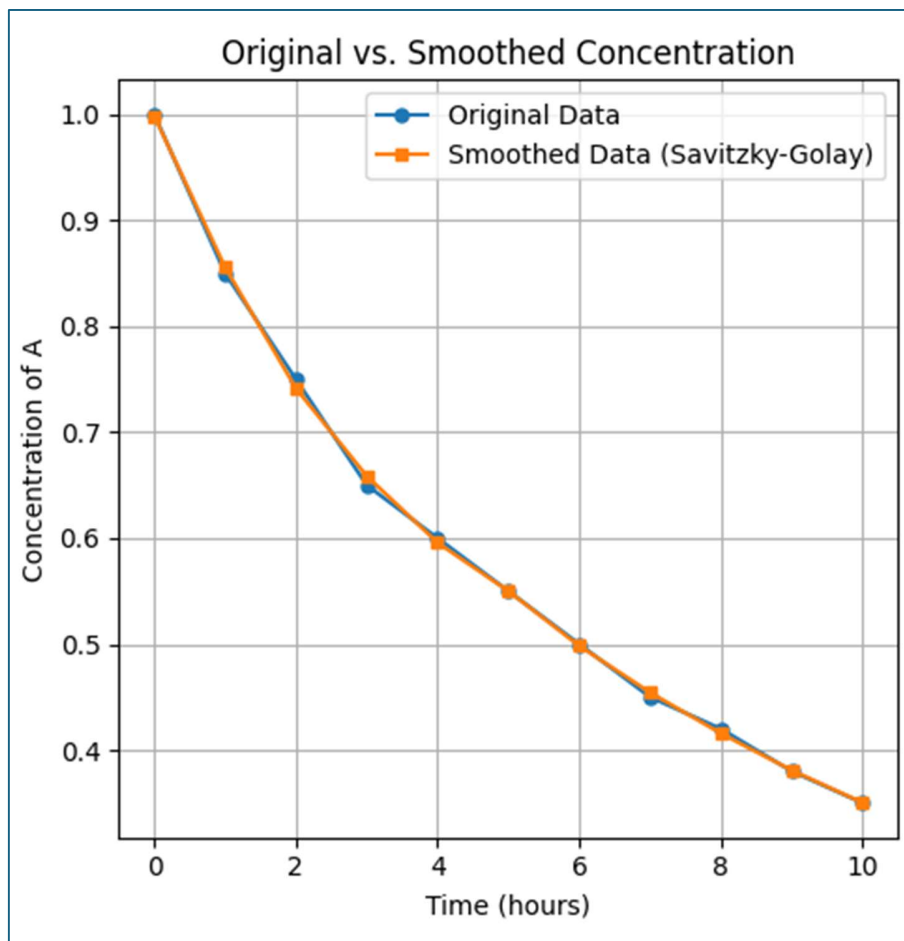


Figure 4 Original vs. Smoothed Concentration of A Over Time

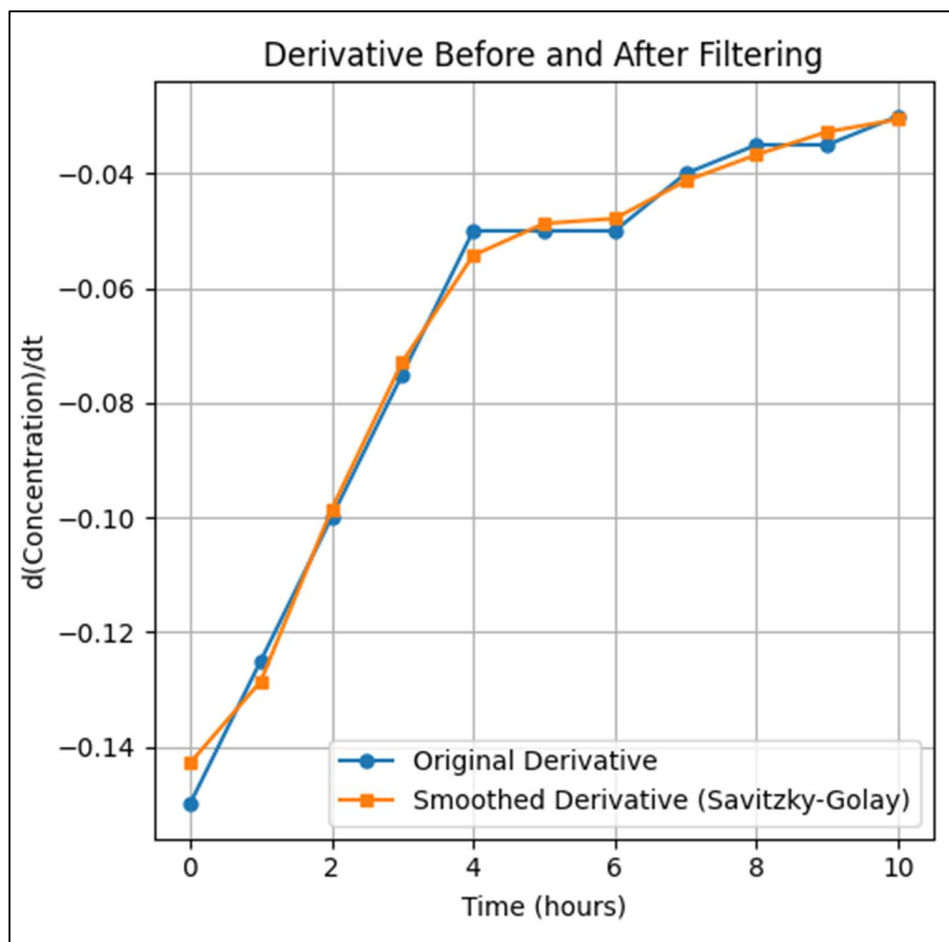


Figure 5 Rate of Change of Concentration (Before and After Filtering)

Key Observations:

- **Concentration vs Time:** The concentration of reactant A decreased over time, which is typical of many chemical reactions, especially those in a batch setting. The data demonstrated a smooth and predictable decline, which is critical for accurate process modeling.
- **Effectiveness of Data Smoothing:** By applying Savitzky-Golay filtering, we removed the noise from the experimental data, allowing for clearer trend analysis. This helped avoid misinterpretation that might have arisen from irregularities in data collection.
- **Polynomial Fitting:** Polynomial fitting allowed us to model the concentration decay more precisely. By adjusting the degree of the polynomial, we observed that a lower-degree polynomial (such as degree 2) captured the broad trend effectively without overfitting the noise, while higher-degree polynomials tended to fit the noise, leading

to overfitting. The balance between underfitting and overfitting was critical to ensure model generalization.

- **Optimization:** Through batch process optimization, we refined the operational parameters to maximize output while minimizing resource consumption. The optimization led to improvements in yield, cost-effectiveness, and efficiency.

Conclusion:

The optimization of the batch process has successfully demonstrated how scientific methods and techniques can be leveraged to enhance industrial operations. Key strategies, such as data filtering, polynomial fitting, and process optimization, have shown how minor changes in process parameters can result in significant improvements in product yield and efficiency.

From an industrial perspective, **the implementation of these methods can lead to:**

1. **Increased Efficiency:** By minimizing inefficiencies such as resource waste and unnecessary energy consumption, we can optimize the batch production processes. This leads to cost reductions and a higher output quality.
2. **Improved Process Monitoring:** Through better trend analysis and noise removal, we can continuously monitor critical parameters such as concentration, temperature, and pressure, leading to more accurate process control.
3. **Scalability:** The methods applied are scalable. Once optimized for laboratory-scale batches, the same principles can be transferred to industrial-scale operations. This ensures that the findings of the research are practical and applicable in real-world industrial settings.
4. **Industry Adoption:** With the optimization techniques and understanding gained from this work, industries can improve the control over their batch processes. This can help them meet production targets with higher precision and consistency, reduce waste, and improve overall profitability.

Moving Forward into Industry:

To transition these results into industry, we need to focus on several key steps:

- **Pilot Testing:** Conduct pilot-scale experiments to validate the findings from the batch optimization study. This will help assess the scalability of the optimization methods under industrial conditions.

- **Automation and Control Systems:** Implement automation systems for continuous data collection and process adjustment. Using techniques such as real-time feedback control, we can dynamically adjust the batch process parameters to ensure optimal performance throughout production.
- **Integration into Industrial Software:** Incorporate the optimization models and data analysis tools into industry-standard software. This can be achieved through collaborations with industrial partners or by using existing process simulation software, like Aspen HYSYS or MATLAB, that is commonly used in the chemical industry.
- **Continuous Monitoring:** Develop real-time monitoring systems based on the optimizations, using sensors and data analytics tools to continuously monitor the process, ensuring consistent product quality and timely adjustments to prevent any deviation from the optimal operating conditions.

By following these steps, industries can effectively implement batch optimization strategies, leading to more sustainable and efficient production practices that not only improve economic performance but also help to meet regulatory and environmental standards.

6. Conclusion

In this report, we have thoroughly explored the Savitzky-Golay filter, its mathematical foundations, and its applications in data smoothing and differentiation. The Savitzky-Golay method is a powerful tool for enhancing data quality, particularly in noisy experimental data, making it especially useful in scientific and engineering applications, including chemical process optimization.

Through the application of Savitzky-Golay filtering on experimental concentration data from a batch process, we demonstrated how the filter can smooth out noise while preserving the underlying trend. This smoothing ability is critical in improving the accuracy of data analysis and subsequent process optimization.

We also explored the importance of selecting an appropriate polynomial degree for the filter. The degree affects the balance between data fitting and overfitting. A lower-degree polynomial effectively captures the general trend, while higher-degree polynomials may result in overfitting, where the model begins to track random fluctuations rather than meaningful trends.

The applications of Savitzky-Golay filtering in real-world industrial processes, such as batch chemical reactions, were discussed, emphasizing its role in improving data accuracy, optimizing process parameters, and reducing noise in sensor readings. The ability to make more precise predictions and decisions based on smoothed data ultimately enhances process efficiency and product quality.

In conclusion, the Savitzky-Golay filter provides a robust and efficient solution to improve data quality in various industrial contexts. Its application in batch process optimization offers clear advantages, including enhanced control over product concentration, improved yield, and optimized resource usage. As industries move toward more automated and data-driven operations, the use of such data-smoothing techniques will become increasingly crucial to ensure higher precision, reliability, and efficiency in production systems.

7. Reference

- [1]** Schmid, M., Rath, D., & Diebold, U. (2022). Why and how Savitzky–Golay filters should be replaced. *ACS Measurement Science Au*, 2(4), 462–472.
<https://doi.org/10.1021/acsmesuresciau.2c00038>
- [2]** Gallagher, N. B. (n.d.). *Savitzky-Golay Smoothing and Differentiation Filter*. Research, Training, and Software. Retrieved from <https://www.eigenvector.com>